

Essa documentação visa descrever claramente os recursos criados, o fluxo de CI/CD e o procedimento de provisionamento, atendendo ao requisito de **documentar quais recursos estão sendo criados e como aplicar a infraestrutura utilizando Terraform do Tech Challenge fase 2**.

Infraestrutura como Código (Terraform)

A infraestrutura da aplicação foi automatizada utilizando Terraform, com provisionamento dos recursos diretamente em um cluster Kubernetes local executado através do Minikube.

O objetivo é garantir que todo o ambiente possa ser criado, atualizado e removido de forma reproduzível e versionada, seguindo os princípios de Infrastructure as Code (IaC).

Arquitetura Provisionada

A infraestrutura criada pelo Terraform é composta pelos seguintes recursos:



Recursos Criados

Namespace Kubernetes

Arquivo:

namespace.tf

Recurso criado:

Namespace: of-fiap

Finalidade:

- Isolar os recursos da aplicação dentro do cluster Kubernetes.
- Facilitar gerenciamento e organização dos componentes.

Secret de Autenticação do GitHub Container Registry (GHCR)

Arquivo:

ghcr-secret.tf

Recurso criado:

Secret: ghcr-secret

Finalidade:

- Armazenar credenciais para autenticação no GitHub Container Registry.
- Permitir que o Kubernetes realize o download das imagens Docker privadas publicadas pela pipeline CI/CD.

Banco de Dados PostgreSQL

Arquivo:

postgres.tf

Recursos criados:

Deployment: postgres

Service: postgres

Finalidade:

- Disponibilizar uma instância PostgreSQL para persistência dos dados da aplicação.
- Expor o banco internamente no cluster através do Service.

Configurações:

- Banco: oficina
- Usuário: postgres
- Porta: 5432

Aplicação Spring Boot

Arquivo:

app.tf

Recursos criados:

Deployment: spring-api

Service: spring-api

Finalidade:

- Executar a API desenvolvida em Spring Boot.
- Realizar o deploy automatizado da imagem Docker gerada pela pipeline.

Configurações:

- Réplicas iniciais: 2
- Porta da aplicação: 8080
- Imagem recebida dinamicamente através da variável:

image_name

A imagem utilizada corresponde à versão gerada pela pipeline CI/CD e identificada através do SHA do commit.

Health Checks

O Deployment da aplicação possui verificações de saúde configuradas através do Spring Actuator.

Endpoints utilizados:

/actuator/health/readiness

/actuator/health/liveness

Finalidade:

- Garantir que apenas instâncias saudáveis recebam tráfego.
- Reiniciar automaticamente containers com falhas.

Pipeline CI/CD

A pipeline foi implementada utilizando GitHub Actions e executada em um GitHub Self-Hosted Runner instalado localmente.

Fluxo de execução:

Build da aplicação
↓
Execução dos testes
↓
Build da imagem Docker
↓
Push para GHCR
↓
Terraform Plan
↓
Terraform Apply
↓
Deploy no Kubernetes
↓
Validação do Rollout

A imagem Docker é publicada utilizando o SHA do commit como tag:

ghcr.io/<usuario>/<repositorio>/spring-api:<commit-sha>

Pré-requisitos

Para execução da infraestrutura são necessários os seguintes componentes instalados:

- Docker Desktop
 - Minikube
 - kubectl
 - Terraform
 - GitHub Self-Hosted Runner (para execução da pipeline)
-

Inicialização do Cluster Kubernetes

Iniciar o Minikube:

```
minikube start --driver=docker
```

Verificar o cluster:

```
kubectl get nodes
```

Resultado esperado:

NAME	STATUS
------	--------

Inicialização do Terraform

Acessar a pasta da infraestrutura:

```
cd terraform
```

Inicializar os providers:

```
terraform init
```

Validar a configuração:

```
terraform validate
```

Gerar plano de execução:

```
terraform plan \  
-var="image_name=<imagem>" \  
-var="ghcr_username=<usuario>" \  
-var="ghcr_token=<token>"
```

Aplicação da Infraestrutura

Executar:

```
terraform apply -auto-approve \  
-var="image_name=<imagem>" \  
-var="ghcr_username=<usuario>" \  
-var="ghcr_token=<token>"
```

Após a execução, o Terraform criará automaticamente:

- Namespace Kubernetes
 - Secret GHCR
 - PostgreSQL
 - Service PostgreSQL
 - Spring API
 - Service Spring API
-

Validação da Implantação

Verificar os recursos:

```
kubectl get all -n of-fiap
```

Exemplo de saída:

```
deployment.apps/postgres  
deployment.apps/spring-api
```

```
service/postgres  
service/spring-api
```

```
pod/postgres-xxxxx  
pod/spring-api-xxxxx
```

Verificar o status do deploy:

```
kubectl rollout status deployment/spring-api -n of-fiap
```

Remoção da Infraestrutura

Para remover todos os recursos criados pelo Terraform:

```
terraform destroy -auto-approve \  
-var="image_name=<imagem>" \  
-var="ghcr_username=<usuario>" \  
-var="ghcr_token=<token>"
```

Todos os recursos provisionados serão removidos automaticamente do cluster Kubernetes.